Quest 3: The ISOlation

Lord of the Wings · counting elven votes written in ISO standards

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: each vote is a number written in an elven positional system where every digit has its own base; convert every vote to decimal and add them up.
Input: one vote per line, such as `ISO-3210 7654`
Output: the candidate's total score in decimal.
What changes between parts: II lets the standard be shorter or longer than the vote; III throws out votes that aren't the smallest way of writing their value.

1. The story

In the Elven Capital, the elves are electing three members of their Grand Council, and the vote count has stalled. Elves don't write numbers in plain decimal. They use many **ISO standards**, each its own positional number system, and all of them count as equally standard. You help convert the votes.

2. Understanding the question

Ordinary place value, as a reminder

In decimal, `7654 = 7×10³ + 6×10² + 5×10¹ + 4×10⁰`. Every position uses the same base, 10, raised to its position counted from the right, starting at 0.

The elven twist

In `ISO-3210 7654`, the **standard** `3210` gives a **separate base for each position**, lined up digit by digit with the vote `7654`:

```
standard:   3     2     1     0
vote:       7     6     5     4
power:      3     2     1     0        (position from the right)

value = 7×3³ + 6×2² + 5×1¹ + 4×0⁰ = 189 + 24 + 5 + 4 = 222
```

In general: `value = Σ vote[i] × standard[i] ^ (length − 1 − i)`.

Things to notice

- **`0⁰ = 1`.** A base of 0 in the last position still counts the digit once (the `+ 4` above). JavaScript's `0 ** 0` is also 1, so the code handles this without any special case.
- **A standard of all one digit is an ordinary base.** `ISO-2222 1011` is binary: 8 + 0 + 2 + 1 = 11. `ISO-8817` is octal: 1×8 + 7 = 15.
- **Base 1** makes every position worth 1, so `ISO-1111 1111` = 1 + 1 + 1 + 1 = 4.

- **Vote digits can be bigger than their base**, for example 7 in a base-3 position. That is what makes Part III necessary.

3. Part I: standard and vote have the same length

Worked examples

```
ISO-3210 7654   7×3³ + 6×2² + 5×1¹ + 4×0⁰ = 189 + 24 + 5 + 4  =  222
ISO-8742 2468   2×8³ + 4×7² + 6×4¹ + 8×2⁰ = 1024 + 196 + 24 + 8 = 1252
ISO-1111 1111   1×1³ + 1×1² + 1×1¹ + 1×1⁰ = 1 + 1 + 1 + 1      =    4
                                                    total     = 1478
```

Step-by-step solution

1. Remove `ISO-` and split the line into the standard (e.g. `3210`) and the vote (e.g. `7654`).
2. Go through the vote's digits from left to right. For digit number `i`, its power is `length - 1 - i`.
3. Multiply the vote digit by (standard digit) ^ (power) and add it to a running total.
4. That total is the vote's decimal value. Add up every vote's value to get the answer.

My solution

```
10  const getAnswer = (input: string) => {
11    const [first = "", second = ""] = input.replace("ISO-", "").split(" ");
12    let answer = 0;
13
14    for (let i = 0; i < first.length; i++) {
15      answer += Number(second[i]) * Number(first[i]) ** (first.length - 1 - i);
16    }
17
18    return answer;
19  };
20
21  let sum = 0;
22
23  INPUTS.forEach((i) => {
24    sum += getAnswer(i);
25  });
26
27  console.log(sum);
```

q3/part1.ts, lines 10–27

Lines	What happens
11	Removes the `ISO-` prefix and splits on the space. `first` is the standard (`"3210"`) and `second` is the vote (`"7654"`), both kept as strings so each digit can be read by index.
14	Walks the positions from left to right.
15	Adds `vote digit × base ^ power`, where the power is `length - 1 - i`: 3, 2, 1, 0 for a 4-digit number.
21–25	Adds up all votes and prints the total.

Trace: `ISO-8742 2468`

i	second[i]	first[i]	power	term
0	2	8	3	2 × 512 = 1024
1	4	7	2	4 × 49 = 196
2	6	4	1	6 × 4 = 24
3	8	2	0	8 × 1 = 8
answer				**1252**

4. Part II: the standard repeats to fit

The question

The standard and the vote can now have different lengths. The standard is a **repeating pattern**: repeat it **from right to left** until it covers the whole vote. The rightmost digits always line up.

```
ISO-123  756423     →  ISO-123123   756423
ISO-123  7564231    →  ISO-3123123  7564231     (an extra 3 on the left)
ISO-2    12345678   →  ISO-22222222 12345678
ISO-2341 945        →  ISO-341      945         (longer standard: keep the rightmost 3)
```

Worked examples (sample)

Vote	Expanded standard	Calculation	Value
`ISO-3210 7654`	3210	as Part I	222
`ISO-8742 2468`	8742	as Part I	1252
`ISO-1 1111`	1111	1+1+1+1	4
`ISO-13 777`	313	$7\times3^2 + 7\times1^1 + 7\times3^0 = 63+7+7$	77
`ISO-34 140856`	343434	$1\times3^5 + 4\times4^4 + 0\times3^3 + 8\times4^2 + 5\times3^1 + 6\times4^0$	1416
`ISO-789 2465112`	9789789	$2\times9^6 + 4\times7^5 + 6\times8^4 + 5\times9^3 + 1\times7^2 + 1\times8^1 + 2\times9^0$	1158390
`ISO-2341 945`	341	$9\times3^2 + 4\times4^1 + 5\times1^0 = 81+16+5$	102
Total			**1161463**

Step-by-step solution

1. Split the line into standard and vote, as in Part I.
2. If their lengths differ, build an expanded standard the same length as the vote. Start from the rightmost digit and walk the standard backwards, wrapping around (`i % length`), putting each digit in front.
3. Compute the value with the expanded standard, exactly as in Part I.
4. Add up every vote's value.

My solution

```
10  const getAnswer = (input: string) => {
11    const [first = "", second = ""] = input.replace("ISO-", "").split(" ");
12
13    let answer = 0;
14    let updatedFirst = "";
15
16    if (first.length !== second.length) {
17      for (let i = 0; i < second.length; i++) {
18        updatedFirst =
19          first[first.length - 1 - (i % first.length)] + updatedFirst;
20      }
21    } else updatedFirst = first;
22
23    for (let i = 0; i < second.length; i++) {
24      answer +=
25        Number(second[i]) * Number(updatedFirst[i]) ** (second.length - 1 - i);
26    }
27
28    return answer;
29  };
```

q3/part2.ts, lines 10–29 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
14	`updatedFirst` will hold the standard after it has been stretched or trimmed to the vote's length.
16	Only builds it if the lengths differ; otherwise uses `first` as it is (line 21).
17–19	Builds the expanded standard **from the right**. For step `i` (0 = rightmost), it takes standard digit `first.length - 1 - (i % first.length)`, which cycles through the standard backwards, and puts it in **front** of what it has so far. This one loop handles both repeating and trimming.
23–25	Same sum as Part I, but over the vote's length and using `updatedFirst`.

Trace: expanding `ISO-789` for `2465112` (7 digits)

i	i % 3	index 2 – (i % 3)	digit	updatedFirst
0	0	2	9	9
1	1	1	8	89
2	2	0	7	789
3	0	2	9	9789
4	1	1	8	89789
5	2	0	7	789789
6	0	2	9	**9789789**

For `ISO-2341 945` the loop runs only 3 times and picks 1, 4, 3, giving `341`. The leading 2 is never used.

5. Part III: ISO2, only the smallest representation counts

The question

Because vote digits can be bigger than their base, **one value can be written in several ways**. With standard `5322`, each of these is 137:

```
1028 → 1×5³ + 0×3² + 2×2¹ + 8×2⁰ = 125 + 0 + 4 + 8 = 137
1036 → 125 + 0 + 6 + 6 = 137        (one "2" moved from the last place to the one before)
1044, 1052, 1060, 1103 → also 137
```

ISO2 removes the ambiguity: a vote is **valid only if no smaller number gives the same value** under the same standard. Here only `1028` is valid. Invalid votes count as 0.

The smaller number can have **fewer digits**. In that case the standard is repeated or trimmed to the shorter length, using the same rule as Part II.

Worked examples (sample)

Vote	Value	Smaller twin	Why	Counted
`ISO2-3 23`	9	`9`	standard 33: 2×3 + 3 = 9; and 9 with standard 3: 9×1 = 9	0
`ISO2-61 149`	34	`54`	standard 161: 1 + 24 + 9 = 34; and 54 with standard 61: 30 + 4 = 34	0
`ISO2-978 99`	72	none		72
`ISO2-5887 5923`	1220	none		1220
`ISO2-76988 123749`	38826	none		38826
`ISO2-435657 929977`	10869	none		10869
`ISO2-3949 870737`	54705	`280207`	same value, smaller number	0
Total				**50987**

Step-by-step solution

1. Compute the vote's value, exactly as in Part II.
2. For every smaller number `i` = 0, 1, 2, … up to vote − 1, compute its value under the **same** standard. The standard is stretched or trimmed to `i`'s length.
3. If any `i` gives the same value, the vote is not the smallest representation: it is invalid and counts as 0. You can stop at the first match.
4. If no smaller number matched, the vote is valid and its value counts.
5. Add up the counted values.

My solution

```
10  const getAnswer = (input: string, needChecking: boolean = true) => {
11    const [first = "", second = ""] = input.replace("ISO2-", "").split(" ");
12
13    let answer = 0;
14    let updatedFirst = "";
15
16    if (first.length !== second.length) {
17      for (let i = 0; i < second.length; i++) {
18        updatedFirst =
19          first[first.length - 1 - (i % first.length)] + updatedFirst;
20      }
21    } else updatedFirst = first;
22
23    for (let i = 0; i < second.length; i++) {
24      answer +=
25        Number(second[i]) * Number(updatedFirst[i]) ** (second.length - 1 - i);
26    }
27
28    if (needChecking) {
29      for (let i = 0; i < Number(second); i++) {
30        if (answer === getAnswer(`ISO2-${first} ${i}`, false)) return 0;
31      }
32    }
33
34    return answer;
35  };
```

q3/part3.ts, lines 10–35 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
10	`getAnswer` takes a second argument, `needChecking`. It is `true` for real votes and `false` when the function calls itself to evaluate a candidate twin, so the check doesn't recurse forever.
11	The prefix is now `ISO2-`.
13–26	Same as Part II: compute the vote's value into `answer`.
29	Brute force: try every smaller number `i = 0, 1, 2, …, vote - 1`.
30	Evaluates `i` under the **same standard** by building the string `` `ISO2-${{first}} ${{i}}` `` and calling itself with checking turned off. `i` is written without leading zeros, so a short `i` gets a stretched or trimmed standard, as in Part II. If the values match, the vote is invalid and the function returns **0**.
34	No smaller twin was found, so the vote is valid and its value is returned.

Trace: `ISO2-3 23` (value 9)

i	Expanded standard	Value of i	== 9?
0	3	0	no
1	3	1	no
…			
8	3	8	no
9	3	9	**yes → return 0**

Single-digit numbers use only the $3^0 = 1$ place, so their value is just the digit, and 9 matches.

How much work does this do?

Vote	Loop stops at	Calls to getAnswer
`ISO2-3 23`	i = 9 (twin found)	10
`ISO2-61 149`	i = 54 (twin found)	55
`ISO2-978 99`	end (valid)	99
`ISO2-3949 870737`	i = 280207 (twin found)	280,208
`ISO2-435657 929977`	end (valid)	929,977

Valid votes check every number below them, so the cost grows with the size of the vote. The puzzle says the numbers in this set are "reasonably small", so this is fast enough. The loop also stops at the **first** twin, which is enough, because one smaller twin already makes the vote invalid.

6. Summary

Part	New rule	Code change
I	each digit has its own base	sum of digit × base^power
II	standard repeats from the right	build `updatedFirst` with `i % first.length`
III	only the smallest representation is valid	try every smaller number by calling itself with `needChecking = false`